

自编码器（Auto-encoder）基本概念

Auto-encoder, Bottleneck, Denoising and BERT



基于李宏毅老师《机器学习 2025》课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 19 分 00 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=28>

目录

1	自编码器在自督导学习中的位置	3
1.1	为什么它也算 self-supervised	3
1.2	名词不必过度纠结	4
1.3	本章小结	4
2	基本架构：编码、瓶颈与重建	4
2.1	形式化目标	5
2.2	重建不是最终目的	5
2.3	和 Cycle GAN 的相似处	5
2.4	瓶颈：表示学习的关键	6
2.5	和 PCA、t-SNE 的关系	7
2.6	本章小结	8
3	为什么低维表示可能有效	8
3.1	高维空间并不等于真实变化很多	8
3.2	课程中的类比：复杂表象背后的少数控制因素	9
3.3	表示压缩和下游资料量	9
3.4	潜在风险：重建目标不一定等于语义目标	10
3.5	本章小结	10
4	自编码器不是新想法：从 Hinton 2006 看历史脉络	10
4.1	当时为什么需要特别的训练流程	11
4.2	今天哪些旧约束已经不重要	11
4.3	历史意义	12
4.4	本章小结	12
5	去噪自编码器与 BERT	12
5.1	去噪目标	12
5.2	BERT 可以看成去噪自编码器	13
5.3	decoder 不一定只是 linear	14
5.4	本章小结	15
6	总结与延伸	15
6.1	本讲核心压缩	16
6.2	和前面课程的连接	16
6.3	继续往下学时要抓住的问题	16

1 自编码器在自督导学习中的位置

前几讲已经看到，BERT、GPT、wav2vec、SimCLR、BYOL 等方法虽然形式差异很大，但都围绕同一件事：不用人工标注，仍然从资料本身制造学习信号。自编码器也属于这条脉络，只是它比 BERT 和 GPT 更早出现，目标也更直观：把输入压成一个中间表示，再从这个表示尽量还原原输入。

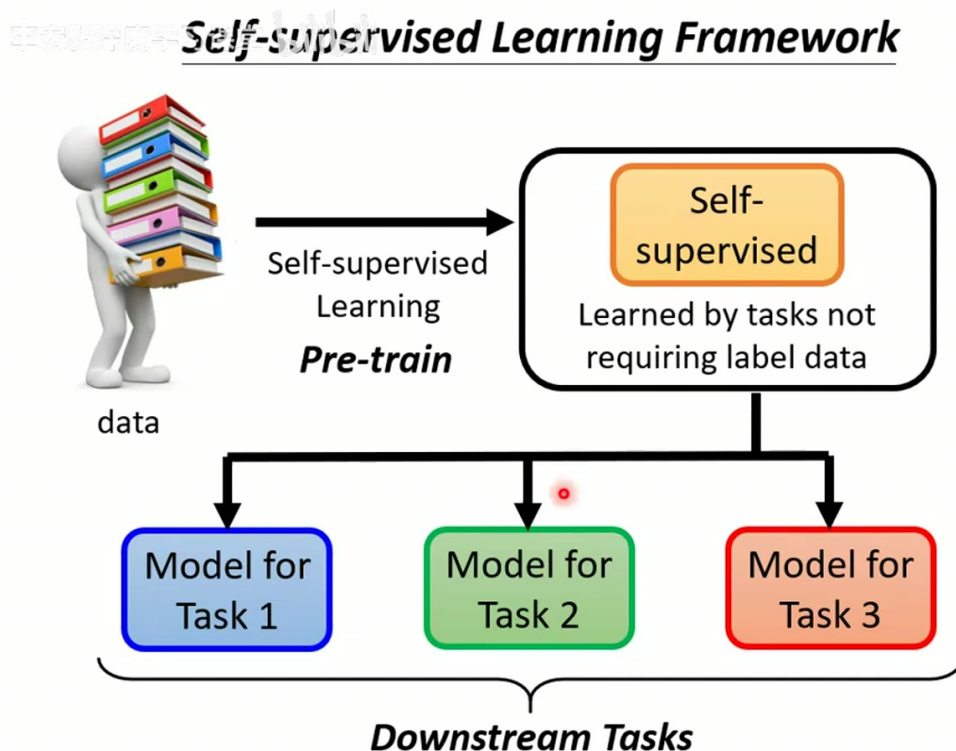


图 1: 自编码器可以放入自督导学习框架：先用不需要标注的任务预训练，再把学到的表示迁移到下游任务¹

从“是否需要人工标签”这个角度看，自编码器是自督导学习的一种。训练时没有人告诉模型这张图片属于哪一类、这个声音对应哪一个字、这个句子表达什么情绪；监督信号来自资料自己。输入本身就是目标，模型要学会先编码、再解码，并让输出接近输入。

1.1 为什么它也算 self-supervised

自督导学习的关键不是“模型一定要做填空题”，而是“训练目标不依赖人工标注”。BERT 的目标是根据上下文猜被遮住的 token；GPT 的目标是预测下一个 token；自编码器的目标是重建输入。三者的监督信号都能从资料本身自动产生。

自编码器的预训练观点

自编码器训练出来的 encoder 本身不一定直接完成某个最终任务。它的价值在于把原始输入转成 embedding、representation 或 code。这个中间表示可以被下游模型接着使用，例如分类、检索、聚类、生成或其他监督式任务。

这也解释了为什么课程一开始把 auto-encoder 放在 self-supervised learning framework 里。自编码器并不是“为了重建图片而重建图片”这么窄；真正想拿走的是中间表示。只要这个表示保留

¹ 视频画面时间区间：00:01:15–00:01:30。

了对下游任务有用的因素，并去掉部分低价值细节，它就可能比原始像素更容易学习。

1.2 名词不必过度纠结

不同教材会把 auto-encoder 归入 unsupervised learning、self-supervised learning、representation learning 或 pretraining。课程中特别提醒：这些边界有时是见仁见智的。若强调“没有人工标签”，它可以叫 unsupervised；若强调“标签由输入本身构造”，它可以叫 self-supervised；若强调“训练一个可迁移的中间表示”，它又是 representation learning。

四个说法的侧重点

Unsupervised learning 强调没有人工标签；self-supervised learning 强调监督信号由资料自动生成；representation learning 强调学到可用的中间特征；pretraining 强调先在大规模资料上训练，再迁移到下游任务。自编码器恰好横跨这些说法。

1.3 本章小结

自编码器不是突然冒出来的独立技巧，而是自督导学习的一种早期形态。它的训练任务是重建输入，训练产物是 encoder 产生的中间表示。理解这一点后，后面所有细节都会更自然：为什么要有瓶颈、为什么要压缩、为什么压缩后的表示能帮助下游任务，以及为什么 BERT 也可以被重新理解为一种去噪自编码器。

2 基本架构：编码、瓶颈与重建

最基本的自编码器由两个神经网络组成：encoder 和 decoder。Encoder 读入原始资料，把它变成一个向量；decoder 读入这个向量，再输出一份和原始资料尽量接近的重建结果。以图片为例，输入可以是一张高维图片，输出仍然是一张图片，中间的向量则是模型学出来的表示。

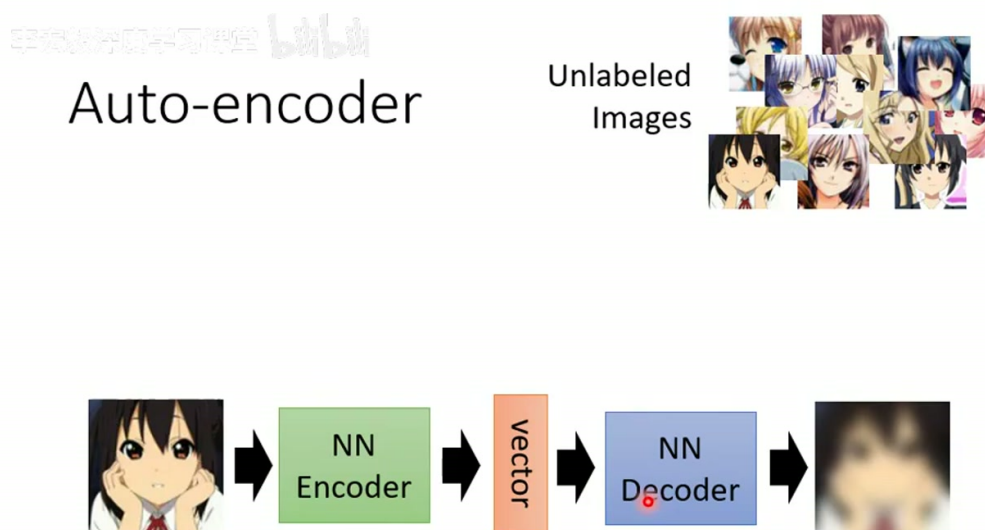


图 2: 基本自编码器结构：输入图片经 NN encoder 变成向量，再由 NN decoder 重建为图片²

2.1 形式化目标

设输入为 x ，encoder 为 f_θ ，decoder 为 g_ϕ 。自编码器可以写成：

$$z = f_\theta(x), \quad \hat{x} = g_\phi(z), \quad \min_{\theta, \phi} \mathcal{L}_{\text{rec}}(x, \hat{x})$$

符号说明：

- x ：原始输入，可以是图片、语音特征、文字序列或其他资料。
- z ：encoder 输出的中间向量，也常被称为 embedding、representation、code。
- $\hat{x}$ ：decoder 根据 z 重建出来的输出。
- $\mathcal{L}_{\text{rec}}$ ：重建损失，用来衡量 $\hat{x}$ 和 x 的差距。

如果输入是连续像素，常见的重建损失可以是 mean squared error 或类似距离；如果输入是离散 token，则可用 cross entropy。课程这一讲主要用图片说明，因此直觉上就是“重建图像和原图越像越好”。

2.2 重建不是最终目的

从表面看，自编码器像是在训练一个“复制机”：输入一张图片，输出同一张图片。若模型只是在像素层面机械复制，那它的学习价值很有限。真正关键在中间的 z 。因为资料必须穿过 z 才能被还原，所以模型被迫把输入中对重建有用的信息组织成一个向量表示。

不要把自编码器理解成单纯压缩软件

压缩软件关心的是尽可能无损地存储与还原档案；自编码器关心的是学到有用表示。若瓶颈太宽、模型容量太大，模型可能近似学会恒等映射，表示不一定有抽象性。若瓶颈太窄或训练目标设计不当，又可能丢掉下游任务真正需要的信息。

2.3 和 Cycle GAN 的相似处

课程用 Cycle GAN 做类比。Cycle GAN 中有两个转换：从 domain X 到 domain Y ，再从 Y 转回 X ，并希望转两次以后能接近原图。自编码器也有类似精神：输入经过 encoder 和 decoder 两次转换后，应尽量回到原来的样子。

²视频画面时间区间：00:03:30–00:03:45。

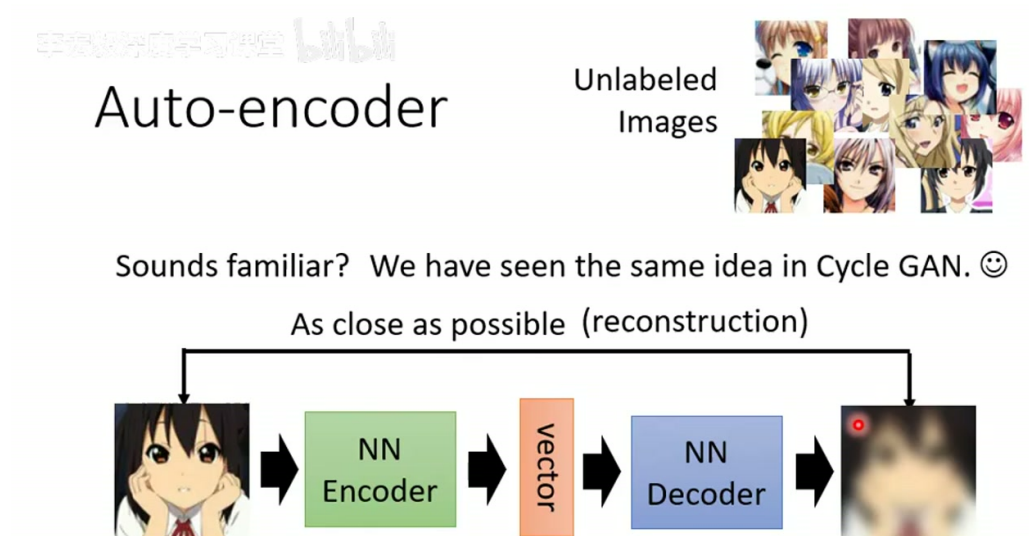


图 3: 自编码器的重建目标和 Cycle GAN 的 cycle consistency 类似: 经过两次转换后尽量回到原输入³

这个类比有助于抓住“无标签训练”的来源。Cycle GAN 不需要成对标注的图片，也能透过循环一致性构造约束；自编码器不需要类别标签，也能透过重建构造约束。两者都把资料结构变成训练信号。

2.4 瓶颈：表示学习的关键

如果 z 的维度和输入一样大，甚至比输入还大，模型可能轻易把所有细节原样搬过去。自编码器常常会让 z 比输入低维得多，使资料经过一个 bottleneck。输入图片可能有上万甚至数十万维像素，而 z 可能只有几十或几百维。

³视频画面时间区间：00:04:45–00:05:00。

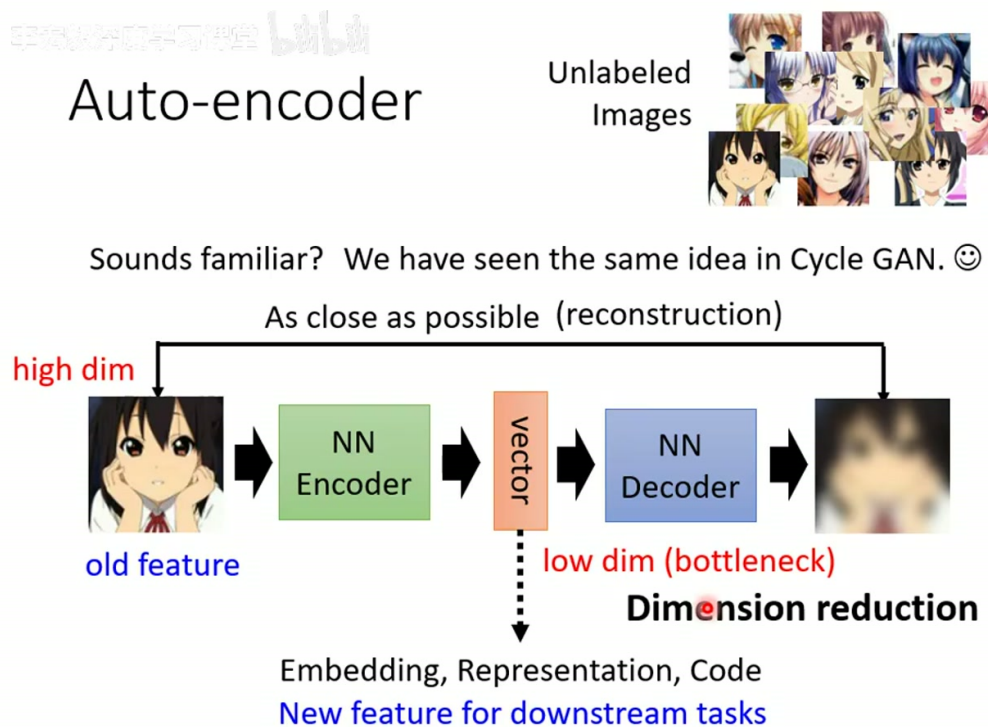


图 4: encoder 将高维旧特征压成低维 bottleneck; 这个 bottleneck 也就是可供下游任务使用的新特征⁴

瓶颈的作用不是故意刁难模型，而是迫使它做信息选择。若它还想重建输入，就必须把最重要、最可解释输入变化的因素放入 z 。因此 z 可能成为比原始像素更好的特征：维度更低、噪声更少、结构更集中，也更容易交给下游监督式模型使用。

瓶颈带来的两种效果

第一，它执行 dimension reduction，把高维输入变成低维表示。第二，它执行 representation learning，让模型自己决定哪些因素值得保留。好的瓶颈不是把所有信息平均压扁，而是把资料变化背后的重要因素抽出来。

2.5 和 PCA、t-SNE 的关系

课程指出，dimension reduction 是一个很大的主题，除了深度学习版本的自编码器，还有 PCA、t-SNE 等传统方法。PCA 通过线性投影保留最大方差方向；t-SNE 更常用于把高维资料可视化到低维平面；自编码器则用神经网络学习非线性的压缩与重建。

⁴ 视频画面时间区间：00:07:30–00:07:45。



More Dimension Reduction

(not based on deep learning)



图 5: 除了自编码器, PCA 和 t-SNE 也是常见降维方法; 课程这里只聚焦深度学习版本⁵

方法	核心机制	更适合的理解方式
PCA	线性投影, 保留主要方差方向	线性降维、快速基线、可解释性较强
t-SNE	保留局部邻近关系, 常用于二维可视化	观察资料分布, 不常作为通用特征抽取器
Auto-encoder	神经网络编码与解码, 最小化重建误差	非线性表示学习, 可接入复杂模型与任务

2.6 本章小结

自编码器的基本形式很简单: encoder 把输入变成 z , decoder 从 z 重建输入。真正值得关注的是 z 。瓶颈让 z 不能无脑携带所有像素细节, 于是模型必须学习压缩资料变化的关键因素。若这些因素正好对下游任务有用, 自编码器就完成了从无标签资料到可迁移表示的转换。

3 为什么低维表示可能有效

课程接着问了一个关键问题: 如果图片原本是高维的, 为什么可以用低维向量表示? 如果一张 3×3 图片需要 9 个数值描述, 甚至 RGB 图片需要 27 个数值, 为什么 encoder 可以把它压到 2 维、10 维或 100 维后还保留重要信息?

3.1 高维空间并不等于真实变化很多

数学上, 一个 3×3 灰阶图像可以看作 9 维空间中的一个点。若每个像素都能自由变化, 可能的图片数量非常巨大。但真实世界图像不会均匀填满整个空间。随机采样一个像素矩阵, 大多数结果

⁵ 视频画面时间区间: 00:07:45–00:08:00。

只是噪声，不像自然图片，也不具有语义结构。

这就是自编码器可行的第一层直觉：资料表面维度很高，但真实资料往往集中在低维结构附近。换句话说，高维像素只是外在坐标；真正的变化因素可能少得多。

低维结构直觉

“低维”不一定表示图片内容简单，而是表示资料变化受约束。人脸图片看似有大量像素，但姿势、光照、表情、身份、背景等因素的组合远少于任意像素矩阵。自编码器试图学习的，就是这些受约束变化背后的表示。

3.2 课程中的类比：复杂表象背后的少数控制因素

课程用《神雕侠侣》中的桥段做类比：表面上变化很多的动作，可能由更少的关键因素控制。对应到图片，复杂像素就像表面的变化；encoder 要找的是背后的控制变量。只要抓住这些变量，就能用更少维度表示看似复杂的资料。

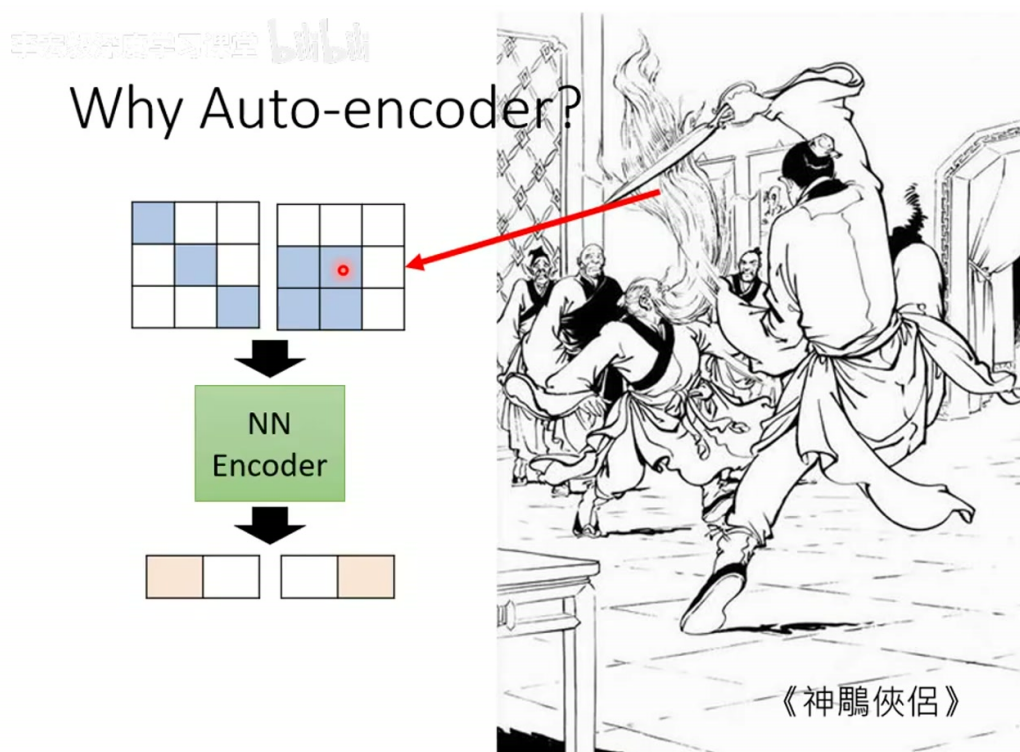


图 6: 课程用“复杂表象由少数因素控制”的类比说明 encoder：找出有限变化后，就能把高维图片压成低维 code⁶

图中把 3×3 的小图片压成 2 维向量，是为了说明一个极端简化情境。若训练资料其实只包含少数几类变化，那么不必用 9 个像素值原样描述每张图；只要记录“属于哪种变化模式”就够了。真实图片当然更复杂，但原则相同：如果资料不是任意分布，而是有规律、有结构、有重复模式，就有被压缩的空间。

3.3 表示压缩和下游资料量

为什么这件事对下游任务有帮助？因为下游模型面对的输入空间变小了。若直接用原始像素训练分类器，模型需要在很高维空间中学习决策边界；若 encoder 已经把图片压成更有结构的 z ，分类

⁶ 视频画面时间区间：00:12:00–00:12:15。

器只需要在表示空间中学习。

这可以降低下游任务对标注资料的需求。未标注图片可能很容易取得，标注图片却昂贵。自编码器先用大量未标注资料学习“图片通常如何变化”，再让少量标注资料负责具体任务，就能把无标签资料的统计结构转化成监督任务中的样本效率。

自编码器帮下游任务的机制

自编码器不是直接替下游任务做分类，而是先把原始资料改写成更适合学习的坐标系。若这个坐标系把无关细节压掉、把重要变化保留，下游模型就更容易用少量标签完成任务。

3.4 潜在风险：重建目标不一定等于语义目标

自编码器的重建损失会鼓励模型保留能还原输入的因素，但“能重建”不保证“对语义任务有用”。例如背景纹理、颜色细节、局部噪声可能对像素重建很重要，却不一定对分类有用；相反，某些高级语义因素对分类重要，但在像素损失里权重未必高。

重建和理解之间有差距

如果只优化像素层面的重建，模型可能过度关注低阶细节。实践中常会加入瓶颈、噪声、稀疏性、变分约束、对抗损失或任务相关目标，让表示更不容易退化成“会复制像素但不理解语义”的编码。

3.5 本章小结

自编码器能做降维，前提是资料本身存在结构：真实资料没有填满整个高维空间，而是沿着较少的因素变化。Encoder 的任务就是找出这些因素，并把复杂输入化成较简单的表示。这个表示可能提升下游样本效率，但重建目标和语义目标并不完全一致，所以自编码器的设计重点在于让 bottleneck、损失函数和资料扰动共同塑造有用表示。

4 自编码器不是新想法：从 Hinton 2006 看历史脉络

课程特别强调，auto-encoder 不是近几年因为大型自督导模型才出现的新概念。早在深度学习重新流行之前，研究者就已经用自编码器做降维和表示学习。一个经典例子是 Hinton、Geoffrey E. 与 Ruslan R. Salakhutdinov 在 2006 年 Science 论文中讨论的用神经网络降低资料维度。

Hinton, Geoffrey E., and Ruslan R. Salakhutdinov. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507

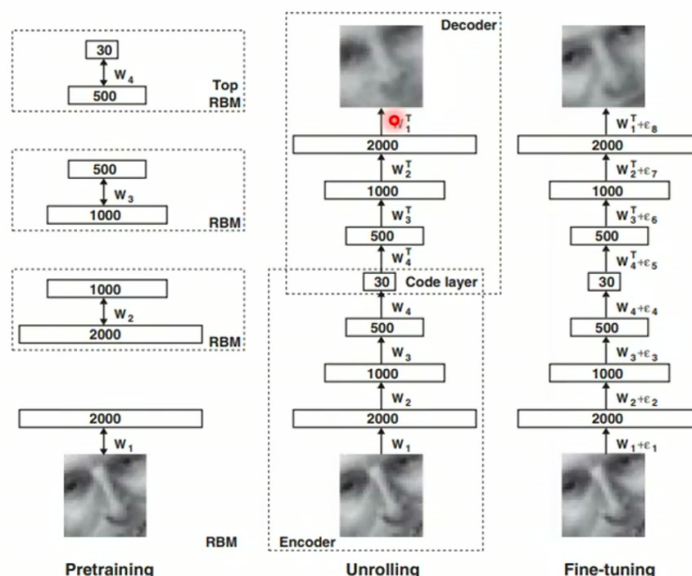


图 7: Hinton 等人在 2006 年已用深层自编码器讨论降维；当时还包含 RBM 预训练、展开和微调等历史做法⁷

4.1 当时为什么需要特别的训练流程

2006 年前后，深层网络还不像今天这样容易训练。现在我们习惯用 ReLU、Batch Normalization、Residual Connection、Adam、大量资料和 GPU 训练深网；但在当时，直接训练很深的神经网络通常很困难。梯度传播、初始化、资料规模和算力都限制了模型深度。

因此早期深度自编码器常采用逐层预训练。课程图中可以看到类似流程：先一层一层训练 restricted Boltzmann machine，再把这些层“展开”成 encoder 和 decoder，最后用反向传播微调整个网络。这个流程今天看起来复杂，但在当时是训练深层表示的一种重要方法。

历史上的三步

Pretraining: 逐层学习初始表示；unrolling: 把逐层模型展开成 encoder-decoder 结构；fine-tuning: 用重建损失微调整个自编码器。这个流程体现了早期深度学习的训练困难，也说明 auto-encoder 曾是深层表示学习的重要入口。

4.2 今天哪些旧约束已经不重要

课程提到，早期还有一些今天较少坚持的信念。例如 encoder 和 decoder 的权重需要对称，decoder 某层权重可能被设成 encoder 对应层权重的 transpose。这样的限制在当时有训练和参数控制上的理由，但今天不一定必要。

现代神经网络训练工具更成熟，encoder 和 decoder 可以是完全不同的架构。图片任务中，encoder 可能是 CNN 或 Vision Transformer，decoder 可能是反卷积网络、Transformer decoder 或扩散模型的一部分；文字任务中，encoder 和 prediction head 也可以按任务需求设计。

⁷ 视频画面时间区间：00:15:45–00:16:00。

4.3 历史意义

理解这段历史有两个好处。第一，它提醒我们不要把“自督导学习”只等同于近年的大模型。自编码器、降维、逐层预训练都是这条线的早期组成。第二，它帮助我们看清技术演化：很多今天看似新颖的模型目标，其实是在旧目标上换了资料模态、网络架构和训练规模。

旧概念的新生命

Auto-encoder 的核心问题一直没有变：如何从未标注资料中学到可用表示。变化的是资料规模、网络结构、优化方法和重建对象。BERT 的 masked language modeling、MAE 的 masked image modeling、去噪扩散模型中的 denoising 目标，都可以和自编码器传统产生联系。

4.4 本章小结

自编码器非常有历史。早期深度自编码器曾依靠 RBM 预训练、unrolling、fine-tuning 和权重对称等技巧来训练；今天这些细节不一定还需要，但“先压缩成表示，再从表示还原”的核心思想仍然活跃。它既是降维方法，也是后来许多自督导目标的概念祖先。

5 去噪自编码器与 BERT

基本自编码器要求 decoder 还原 encoder 看到的输入。去噪自编码器则稍微改一下：先把输入加噪声，让 encoder 看到被破坏的版本，再要求 decoder 还原干净版本。这个变化看似小，却非常重要，因为它迫使模型学习更稳定、更鲁棒的表示，而不是简单复制输入。

5.1 去噪目标

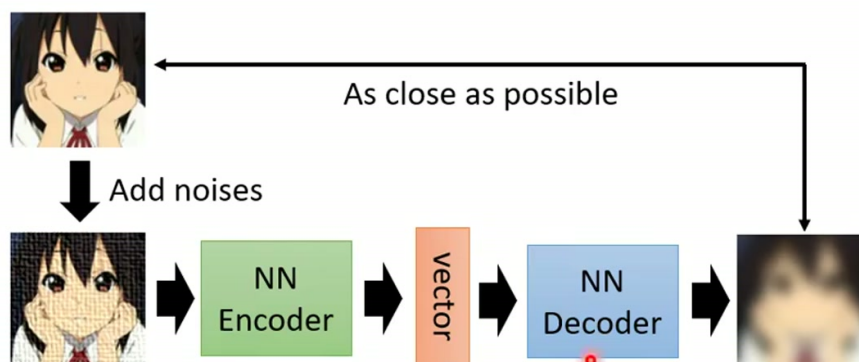
设干净输入为 x ，加噪后的输入为 $\tilde{x}$ ：

$$\tilde{x} = q(x), \quad z = f_{\theta}(\tilde{x}), \quad \hat{x} = g_{\phi}(z), \quad \min_{\theta, \phi} \mathcal{L}_{\text{denoise}}(x, \hat{x})$$

符号说明：

- $q(\cdot)$ ：加噪过程，可以是遮挡、扰动、mask、随机替换或其他破坏方式。
- $\tilde{x}$ ：被破坏后的输入，也是 encoder 实际看到的内容。
- x ：干净目标，decoder 要尽量还原它。
- $\mathcal{L}_{\text{denoise}}$ ：去噪重建损失，衡量输出是否接近干净输入。

De-noising Auto-encoder



The idea sounds familiar? 😊

Vincent, Pascal, et al. "Extracting and composing robust features with denoising autoencoders." *ICML*, 2008.

图 8: 去噪自编码器先对输入加噪, 再让 encoder 和 decoder 联手还原未加噪的原始图片⁸

去噪目标改变了模型学习的压力。若输入没有噪声, 模型可以保留许多低层细节; 若输入已经被破坏, 模型必须根据上下文、结构和资料规律推断缺失或污染部分。这更接近“理解资料分布”, 也更接近现代 masked modeling 的思想。

去噪比普通重建更强的地方

普通重建问的是“能否复制输入”; 去噪重建问的是“在输入不完整或被污染时, 能否根据资料结构恢复合理内容”。后者更能迫使模型利用上下文与统计规律, 因此通常更适合表示学习。

5.2 BERT 可以看成去噪自编码器

课程把 BERT 拉回来做对照: BERT 的输入句子会被 mask 掉一些 token, 这些 mask 就像噪声; BERT 主体把加噪句子编码成 contextual embedding; 上方的 linear 层和 softmax 负责预测被遮住的词; 训练目标是让预测词接近原词。

⁸视频画面时间区间: 00:17:15–00:17:30。

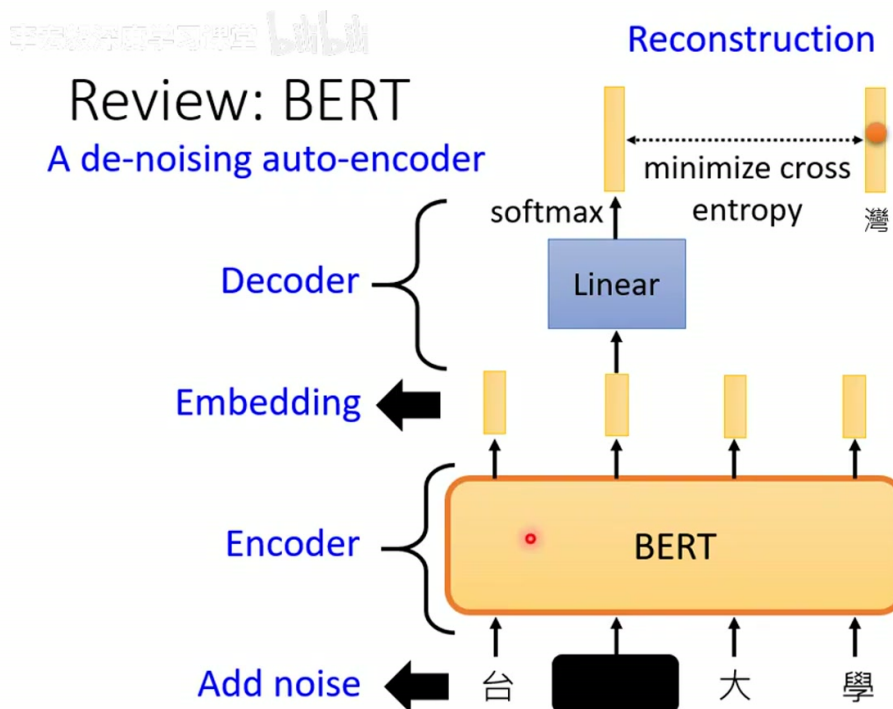


图 9: BERT 可被视为去噪自编码器: mask 是噪声, BERT 是 encoder, 预测头是 decoder, 目标是重建被遮住的 token⁹

用公式写, 若原句为 s , 遮蔽后的句子为 $\tilde{s}$:

$$\tilde{s} = \text{mask}(s), \quad h = \text{BERT}(\tilde{s}), \quad \hat{s}_{\text{mask}} = \text{softmax}(Wh)$$

符号说明:

- s : 原始句子, 例如 “台大机器学习”。
- $\tilde{s}$: mask 后的句子, 例如中间某些 token 被遮住。
- h : BERT 输出的 contextual embedding。
- W : 预测头参数, 可视作一个简单 decoder。
- $\hat{s}_{\text{mask}}$: 模型对被遮住 token 的预测分布。

这就是为什么课程说 BERT 其实可以被看成 denoising auto-encoder。它不是重建整张图片, 而是重建被破坏的文字位置; 它不是用 pixel loss, 而是用 cross entropy; 它的 encoder 是 Transformer, decoder 可以只是 linear prediction head。

5.3 decoder 不一定是 linear

课程进一步提醒: 如果把 BERT 的最后一层输出当作 embedding, 那么上面的 linear 层可以叫 decoder; 但如果把第六层输出当作 embedding, 那么第七到第十二层也可以被视为 decoder 的一部分。因此 “哪一段是 encoder、哪一段是 decoder” 有时取决于你把哪一层表示拿出来用。

这点很重要, 因为它避免我们把自编码器理解成固定形状的绿色方块加蓝色方块。自编码器是一种目标与分解方式: 某段网络把输入编码成表示, 另一段网络从表示还原目标。具体边界可以随架构和使用方式改变。

⁹ 视频画面时间区间: 00:18:15–00:18:30。

不要把图上的结构当作唯一形式

经典图示里的 encoder、vector、decoder 只是最简版本。BERT、MAE、去噪扩散模型或多模态模型中，编码与解码可能共享层、交错出现，或只在训练时存在。判断是否有 auto-encoding 思想，应看训练目标和表示瓶颈，而不是只看图形外观。

5.4 本章小结

去噪自编码器把“复制输入”升级成“从被破坏的输入恢复干净目标”。这个想法直接连接到 BERT：mask token 是加噪，预测原 token 是去噪重建。BERT 的成功说明，自编码器思想不只适用于图片，也能通过合适的破坏方式和重建目标迁移到文字。更广泛地说，许多现代自督导模型都可以看成在设计更好的“破坏”和“重建”。

6 总结与延伸

这一讲到 Basic Idea of Auto-encoder 先告一段落。结尾的 outline 显示，后面还会继续讨论 feature disentanglement、discrete latent representation 和更多应用。这也说明本讲只是打基础：先理解 encoder-decoder、bottleneck、reconstruction、denoising，后面才能讨论更复杂的 latent representation 设计。



Outline

Basic Idea of Auto-encoder

Feature Disentanglement

Discrete Latent Representation

More Applications

图 10: 本讲结束后，课程将继续进入 feature disentanglement、discrete latent representation 与更多应用¹⁰

¹⁰ 视频画面时间区间：00:18:45–00:19:00。

6.1 本讲核心压缩

概念	含义
Auto-encoder	用 encoder 把输入变成 code，再用 decoder 从 code 重建输入。
Reconstruction Bottleneck	训练信号来自输入本身，不需要人工标签。中间表示维度较低，迫使模型选择重要信息。
Dimension reduction	自编码器可被视为一种非线性降维方法，但目标不只是可视化。
Denoising AE	对输入加噪，再要求模型还原干净目标，使表示更鲁棒。
BERT connection	mask token 相当于加噪，预测原 token 相当于去噪重建。

6.2 和前面课程的连接

自编码器把前面讲过的多个主题串在一起。和 self-supervised learning 的连接在于：重建目标自动产生监督信号。和 representation learning 的连接在于：真正想使用的是 encoder 的中间表示。和 Cycle GAN 的连接在于：两次转换后回到原输入的 consistency 约束。和 BERT 的连接在于：mask 后重建原 token 正是去噪自编码器的文字版本。

最重要的一句话

自编码器的价值不在于把输入原封不动复制出来，而在于通过“编码到瓶颈再重建”的压力，让模型从未标注资料中学到更紧凑、更稳定、可迁移的表示。

6.3 继续往下学时要抓住的问题

后续如果进入 feature disentanglement，需要问：code 的不同维度能不能对应到不同因素，例如姿势、颜色、身份、风格？如果进入 discrete latent representation，需要问：连续向量是否一定最好，能不能把表示离散化成类似 token 的结构？如果进入更多应用，需要问：自编码器学到的表示究竟帮助了哪个任务，是生成、压缩、检索、异常检测，还是下游分类？

这些问题都指向同一件事：latent representation 的形状会决定模型能做什么。自编码器只是入口，真正的研究空间在于如何设计表示、如何约束表示、以及如何验证表示对下游任务有价值。